Atlas — Systems Requirements

Multi-Branch · Modular · Single Source of Truth
Version 1.0 · 2026-09-18 · Status: Draft for Engineering

1. Purpose & Scope

This document defines the system-level requirements for Atlas, a multi-branch store operations platform. It covers organizational hierarchy, tenancy, modularity, authorization, core functional domains, data integrity, non-functional requirements, and the modular architecture that allows independent and multi-location retailers to run POS, inventory, staffing, and analytics from one ledger.

In scope: Multi-tenant / multi-branch architecture, role-based access with store-scoped and tenant-scoped permissions, modular feature flags, real-time inventory, POS with offline support, staffing, reporting, audit, and API contracts.

Out of scope for V1: Full e-commerce storefront, marketplace multi-vendor, advanced AI recommendations, cryptocurrency payments, complex international tax engines, real-time chat.

2. Organizational Structure & Multi-Branch Model

Atlas is designed around a two-level isolation model that maps cleanly to real retail organizations.

2.1 Hierarchy

Tenant (Business / Organization) — The legal or commercial entity. One tenant owns one or more stores. All data is isolated at the tenant level. Tenant-level settings (currency defaults, tax regimes, product types, module entitlements) cascade to stores and can be overridden where appropriate.

Store (Branch / Location) — A physical or logical point of sale and inventory location. Each store has its own stock levels, pricing overrides, staff assignments, shifts, and daily takings. Stores are hard-partitioned for operational data while remaining visible to authorized tenant-level roles.

Conceptual structure:

```
Tenant
  ■■■ Settings (currency, tax, modules, branding)
  ■■■ Shared Catalogue (product types, global products)
  ■■■ Users & Role Assignments
  ■■■ Store A (Ikeja)
  ■     ■■■ Inventory, Prices, Staff, Shifts, Sales
  ■■■ Store B (Lekki)
  ■     ■■■ Inventory, Prices, Staff, Shifts, Sales
  ■■■ Store C (...)
```

2.2 Visibility Rules

- **Tenant Admin / Owner** — Can see and act on every store belonging to the tenant. Cross-store dashboards, consolidated reports, user management, module configuration, and inter-store stock transfers are available.
- **Store Manager** — Scoped to one or more explicitly assigned stores. Sees only those stores' inventory, sales, staff, and reports. Cannot access other branches or tenant-level settings.
- **Cashier / Associate** — Scoped to a single store (or the store of the active shift). POS, limited inventory visibility (read-only or stock-count only), no access to other stores or financial reports.
- **Platform Super-Admin** (Atlas internal) — Cross-tenant support and operations; never used by customers.

2.3 Key Requirements — Organizational Model

ORG-001 — Every record that belongs to a store shall carry a non-null store_id and tenant_id.

ORG-002 — Tenant isolation shall be enforced by a global query filter / row-level security; tenant_id is never accepted from the client.

ORG-003 — A user may hold different roles in different stores (e.g. Manager in Store A, Cashier in Store B).

ORG-004 — Tenant Admin role grants read/write visibility across all stores of that tenant.

ORG-005 — Store-scoped roles shall be rejected when the requested resource belongs to a different store.

ORG-006 — Creating a new store shall be restricted to Tenant Admin (or higher).

ORG-007 — Inter-store stock transfers shall be first-class transactions that debit one store and credit another atomically.

ORG-008 — All reports and dashboards shall support both single-store and multi-store (tenant-wide) aggregation based on the caller's effective permissions.

3. Roles, Permissions & Authorization

Authorization is permission-based and always evaluated as (user + store + action). A permission is never global for a multi-store user; it is scoped.

3.1 Core Roles

Role	Scope	Typical Capabilities
Tenant Admin / Owner	All stores of tenant	Full access: users, modules, all stores, financials, settings, transfers
Store Manager	Assigned store(s)	Inventory, staffing, sales, reports, receiving, adjustments for assigned stores
Cashier	Active store / shift	POS, limited stock visibility, returns (if permitted), shift open/close
Stock Clerk	Assigned store(s)	Receiving, counts, adjustments; no sales or financial reports
Viewer / Accountant	Assigned store(s) or tenant	Read-only reports and audit

3.2 Permission Model Requirements

AUTHZ-001 — Every protected endpoint shall declare the required permission and the required scope (tenant or store).

AUTHZ-002 — Role grants are stored as (user_id, role, store_id | null). Null store_id means tenant-wide.

AUTHZ-003 — Permission checks shall fail closed; missing or ambiguous scope results in 403.

AUTHZ-004 — Admin endpoints that operate across stores shall require Tenant Admin (or explicit cross-store grant).

AUTHZ-005 — Audit log shall record actor, action, resource, store_id, tenant_id, and outcome for every privileged operation.

4. Modular Architecture

Atlas is built as a modular monolith. Modules can be enabled or disabled per tenant (and, where useful, overridden per store). Disabled modules are invisible in the UI and their APIs return 404 or a clear "module not enabled" error. This keeps the product simple for single-location shops while allowing multi-location operators to grow into advanced capabilities without migrating systems.

4.1 Core Modules (always present)

- Identity & Access (tenants, stores, users, roles, sessions)
- Catalogue & Product Types
- Inventory & Stock Ledger
- Point of Sale & Payments
- Audit & Observability

4.2 Optional / Toggleable Modules

Module	Description	Typical Enablement
Staffing & Scheduling	Shifts, clock-in/out, labor cost vs sales	Multi-location or larger teams
Purchasing & Suppliers	POs, GRNs, supplier balances	When reordering is formalized
Customers & Loyalty	Customer profiles, purchase history, simple loyalty	Retailers with repeat customers
Advanced Analytics	Demand forecasting, multi-store comparison, shrink	Growth / Multi-Location tier
Inter-Store Transfers	Formal transfer orders between branches	2+ stores
Recipes / BOM	Ingredient depletion for restaurants / production	Food & beverage, production
Serial / Batch Tracking	Per-unit or per-batch identity	Pharmacy, electronics, regulated goods

4.3 Modularity Requirements

MOD-001 — Module enablement is stored at tenant level and may be overridden at store level where the product design allows.

MOD-002 — A disabled module shall not appear in navigation, shall not accept writes, and shall not affect core ledger integrity.

MOD-003 — Core modules (Identity, Catalogue, Inventory, POS, Audit) cannot be disabled.

MOD-004 — New modules shall be added behind feature flags and released without requiring schema migrations that break existing tenants.

MOD-005 — Billing / entitlement checks for paid modules shall be enforced server-side on every relevant request.

5. Functional Requirements by Domain

5.1 Identity, Authentication & Sessions

ID-001 — Users authenticate with email + password (or SSO in later phases). Passwords hashed with Argon2id or equivalent.

ID-002 — Sessions are server-side or short-lived JWT + refresh token; refresh tokens are rotatable and revocable.

ID-003 — A user may belong to multiple tenants only if explicitly invited; default is single-tenant.

ID-004 — Password reset, email verification, and account lockout after repeated failures are required.

ID-005 — Every authenticated request carries an effective (tenant_id, user_id, store_context) derived server-side.

5.2 Store Management

STORE-001 — Tenant Admin can create, update, archive stores.

STORE-002 — Each store has: name, code, address, timezone, currency (inherit or override), tax profile, opening hours, receipt settings.

STORE-003 — Archived stores remain readable for historical reports but cannot accept new sales or stock movements.

STORE-004 — Store-level settings override tenant defaults where the setting is marked overridable.

5.3 Catalogue & Products

CAT-001 — Product types define attribute schemas (JSON schema or equivalent). Attributes can be required, searchable, and identity-bearing (serial/batch).

CAT-002 — Products belong to a product type and carry base unit of measure; additional sale units with conversion factors are supported.

CAT-003 — Quantity is decimal everywhere; never integer-only.

CAT-004 — Pricing is resolved by rules (store, customer group, quantity, date) rather than a single static column.

CAT-005 — SKUs / barcodes are unique within a tenant (or within store if configured).

CAT-006 — Products can be active/inactive; inactive products cannot be sold.

5.4 Inventory & Stock Ledger

INV-001 — Stock is held per (store, product [, batch/serial]). Available = on-hand − reserved.

INV-002 — Every stock change is an immutable ledger entry (receipt, sale, adjustment, transfer, return) with reason and actor.

INV-003 — Sales reserve then consume stock transactionally; concurrent checkouts cannot oversell.

INV-004 — Manual adjustments require a mandatory reason code.

INV-005 — Low-stock thresholds are configurable per product per store; alerts are generated when available falls below threshold.

INV-006 — Inter-store transfers create paired ledger entries that keep total tenant stock consistent.

INV-007 — Stock counts (stocktake) support partial or full counts and produce variance adjustments.

5.5 Point of Sale & Payments

POS-001 — Checkout supports barcode/SKU search, quantity, line and basket discounts, tax, multiple tenders (cash, card, transfer, split).

POS-002 — Offline mode: sales are queued locally with client-generated IDs and device ID; sync is idempotent on reconnect.

POS-003 — Receipts show subtotal, tax, discount, total and enforce total = subtotal + tax − discount.

POS-004 — Returns / voids restore stock and are linked to the original sale.

POS-005 — Shifts: open/close with cash declaration and variance recording.

POS-006 — Payment provider integration is abstracted; webhooks and verification are server-authoritative.

POS-007 — All monetary values use integer minor units (e.g. kobo/cents) or a precise decimal type; never floating-point.

5.6 Staffing & Scheduling (Module)

STAFF-001 — Shifts can be created, published, and assigned to users for a store.

STAFF-002 — Clock-in / clock-out is recorded and can be constrained to scheduled shifts.

STAFF-003 — Labor cost vs sales ratio is computable in real time for a store or tenant.

STAFF-004 — Conflict and overtime warnings are generated when schedules violate rules.

5.7 Reporting & Analytics

RPT-001 — Daily / date-range sales summary (totals, count, by tender, by store).

RPT-002 — Inventory valuation (qty × cost) per product / category / store.

RPT-003 — Low-stock list and top/bottom sellers.

RPT-004 — Multi-store comparison dashboards available only to roles with cross-store visibility.

RPT-005 — All reports respect the caller's store and tenant scope; no data leakage across tenants.

6. Data Integrity, Concurrency & Idempotency

DATA-001 — PostgreSQL is the source of truth. All financial and stock mutations run inside ACID transactions.

DATA-002 — Row-level locking or equivalent is used for inventory reservation to prevent oversell under concurrency.

DATA-003 — Payment webhooks and offline sync endpoints are idempotent; duplicate delivery must not create duplicate sales or stock movements.

DATA-004 — Order / sale items store historical price, tax, and product snapshot so later catalogue changes do not rewrite history.

DATA-005 — Foreign keys, unique constraints, and check constraints are enforced in the database, not only in application code.

DATA-006 — Soft-delete / archive is preferred over hard delete for master data (products, stores, users).

7. Non-Functional Requirements

Category	Requirement	Target
Performance	Checkout path latency (online)	≤ 200 ms p95
Performance	Dashboard / report queries	≤ 2 s for typical multi-store range
Availability	POS / checkout path uptime	99.9%
Resilience	Offline checkout	Full capability; durable local queue; idempotent sync
Security	Payment data	PCI DSS scope minimized; no card data stored if possible
Security	Transport & at-rest	TLS 1.2+; encryption at rest for sensitive fields
Security	Auth	Argon2id (or better); short-lived tokens; refresh rotation
Scalability	Architecture	Modular monolith; horizontal scale of API tier; DB read replicas as needed
Observability	Logging / metrics / tracing	Structured logs, request IDs, key business metrics, health/ready endpoints

Category	Requirement	Target
Compliance	Audit	Immutable audit trail for privileged and financial actions
Usability	Time to first sale	≤ 15 minutes from signup for a new single-store tenant

8. API & Integration Principles

Public and private APIs follow a consistent versioned REST (or RPC) style. All mutating endpoints require authentication and authorization. Store context is either derived from the session or explicitly supplied and then verified against the user's grants.

API-001 — Base path versioned (e.g. /api/v1).

API-002 — Consistent error envelope: { "error": { "code": "...", "message": "...", "details": ... } }.

API-003 — Pagination, filtering, and sorting supported on all list endpoints.

API-004 — Webhooks for payment providers are authenticated and verified; processing is idempotent.

API-005 — OpenAPI / equivalent documentation is generated and kept current.

API-006 — Rate limiting is applied to authentication and other abuse-sensitive endpoints.

9. Security Requirements (Summary)

SEC-001 — HTTPS only in production.

SEC-002 — Passwords never stored in plaintext; modern KDF (Argon2id recommended).

SEC-003 — Authorization is enforced on every request; UI hiding is not security.

SEC-004 — Tenant and store isolation is server-enforced; client-supplied tenant/store IDs are treated as claims to be verified, not trusted.

SEC-005 — Sensitive data (tokens, payment secrets, PII) is never written to application logs.

SEC-006 — Principle of least privilege for service accounts and database roles.

SEC-007 — Dependency and container scanning in CI; secrets managed outside source control.

10. Acceptance Criteria (MVP Multi-Branch)

The system is considered to meet the multi-branch modular bar when:

- A Tenant Admin can create multiple stores and see consolidated and per-store views.
- A Store Manager assigned to Store A cannot read or mutate Store B data.
- Inventory movements and sales are correctly isolated per store while remaining visible in tenant roll-ups for authorized users.
- Modules can be enabled/disabled without breaking core flows.
- Concurrent sales at the same store cannot oversell stock.
- Offline sales sync without duplication.
- Audit trail captures who did what, in which store, under which tenant.
- Core purchase lifecycle (browse → cart/checkout → pay → stock decrement → receipt) works end-to-end for a single store and across stores for admins.

Document control: This Systems Requirements document is the authoritative source for organizational structure, multi-branch behavior, modularity, and cross-cutting system constraints. Feature-level detail lives in the Tech PRD and Product PRD. Changes to tenancy, authorization, or module boundaries require an update to this document.